

1. Sintassi del λ -Calcolo

Il λ -calcolo definisce le funzioni in modo **intensionale**, cioè attraverso un processo computazionale, non come insieme di coppie ordinate.

- **Alfabeto**: un insieme numerabile di variabili $Var = \{x, y, z, \dots\}$.
- **Termini (Λ)**: definiti per ricorsione:
 - Ogni variabile è un termine: $x \in \Lambda$.
 - Se $E \in \Lambda$ e $x \in Var$, allora $\lambda x.E \in \Lambda$ (astrazione).
 - Se $E_1, E_2 \in \Lambda$, allora $(E_1 E_2) \in \Lambda$ (applicazione).

Convenzioni di precedenza per ridurre le parentesi:

- L'applicazione è associativa a sinistra: $E_1 E_2 E_3 \equiv ((E_1 E_2) E_3)$.
- L'astrazione è associativa a destra: $\lambda x.\lambda y.E \equiv \lambda x.(\lambda y.E)$.
- L'applicazione ha priorità sull'astrazione: $\lambda x.E_1 E_2 \equiv \lambda x.(E_1 E_2)$.

Esempi fondamentali:

- $\lambda x.x$: funzione identità.
- $\lambda x.\lambda y.x$: proiezione del primo argomento.
- $\lambda x.\lambda y.y$: proiezione del secondo argomento.

2. Variabili libere, legate e sostituzione

Sottotermini

L'insieme dei sottotermini $st(E)$ è definito induttivamente:

- $st(x) = \{x\}$
- $st(\lambda x.E) = \{\lambda x.E\} \cup st(E)$
- $st(E_1 E_2) = \{E_1 E_2\} \cup st(E_1) \cup st(E_2)$

Variabili occorrenti, libere e legate

- Un'occorrenza di x è **legata** se si trova nel corpo di un λx ; altrimenti è **libera**.
- $var(E)$: insieme di tutte le variabili che compaiono in E .
- $varlib(E)$: insieme delle variabili con almeno un'occorrenza libera.
- $varleg(E)$: insieme delle variabili con almeno un'occorrenza legata.

Nota: $var(E) = varleg(E) \cup varlib(E)$, ma i due insiemi possono intersecarsi (es. $(\lambda x.x) x$).

- **Termine chiuso:** $varlib(E) = \emptyset$.
- **Termine aperto:** $varlib(E) \neq \emptyset$.

Sostituzione $E[F/x]$

Sostituisce F a ogni occorrenza **libera** di x in E .

La definizione è complessa perché deve evitare la **cattura di variabili**:

- Se $E = \lambda x. E'$ (legatore omonimo), la sostituzione non entra nel corpo.
- Se $E = \lambda y. E'$ con $y \neq x$, e y non è libera in F , si sostituisce normalmente.
- Se y è libera in F , si deve prima rinominare y in E' con una variabile z fresca, poi sostituire.

Esempio: $(\lambda x. x \ y)[x/y]$ non diventa $\lambda x. x \ x$ (errato), ma dopo α -rinominazione:

$$\lambda z. (z \ y)[x/y] = \lambda z. z \ x.$$

3. Regole di conversione (semantica)

Il λ -calcolo ha tre regole di riscrittura:

1. **α -conversione:**

$$\lambda x. E =_{\alpha} \lambda y. (E[y/x]), \text{ se } y \notin \text{varLibere}(E).$$

Permette di rinominare variabili legate senza cambiare il significato.

2. **β -conversione** (regola fondamentale):

$$(\lambda x. E) \ F =_{\beta} E[F/x].$$

Formalizza l'applicazione di una funzione a un argomento.

3. **η -conversione** (estensionalità):

$$\lambda x. E \ x =_{\eta} E, \text{ se } x \notin \text{varLibere}(E).$$

Esprime che due funzioni sono uguali se producono gli stessi risultati su ogni argomento.

Queste regole generano relazioni di equivalenza ($=_{\alpha}$, $=_{\beta}$, $=_{\eta}$) che sono **congruenze** rispetto a sintassi e applicazione.

4. Riduzione e strategie

Le regole possono essere orientate come **riduzioni**:

- **β -riduzione:** $(\lambda x. E) \ F \rightarrow_{\beta} E[F/x]$
- **η -riduzione:** $\lambda x. E \ x \rightarrow_{\eta} E$ (se x non libera in E)

Forma normale

Un termine è in **forma normale** se non contiene β -radicali, cioè nessun sottotermine della forma $(\lambda x. E) \ F$. Non tutti i termini hanno forma normale (es. $\Omega = (\lambda x. x \ x)(\lambda x. x \ x)$ che riduce all'infinito).

Strategie di riduzione

- **Call-by-name:** riduce sempre il radicale più esterno a sinistra.
 - Se la forma normale esiste, questa strategia la trova (teorema di Curry).
- **Call-by-value:** riduce il radicale più interno a sinistra (prima valuta gli argomenti).
 - Più efficiente in alcuni casi, ma può non terminare anche se la forma normale esiste.
- **Call-by-need** (lazy evaluation): condivide i sottotermini per evitare ricalcoli, combinando i vantaggi delle due precedenti.

Confluenza (Teorema di Church-Rosser)

- Se $E \rightarrow_{\beta}^* E_1$ e $E \rightarrow_{\beta}^* E_2$, allora esiste E' tale che $E_1 \rightarrow_{\beta}^* E'$ e $E_2 \rightarrow_{\beta}^* E'$.
- **Conseguenza:** la forma normale, se esiste, è unica (a meno di α -conversione).
- La β -teoria è **coerente**: non tutti i termini sono equivalenti.

5. Logica Combinatoria

La logica combinatoria è un formalismo equivalente al λ -calcolo ma **senza variabili legate**.

Si basa su due soli combinatori:

- $K = \lambda x. \lambda y. x$
- $S = \lambda x. \lambda y. \lambda z. x z (y z)$

Regole di riduzione:

- $K P Q \rightarrow P$
- $S P Q R \rightarrow P R (Q R)$

Simulazione della λ -astrazione

Si definisce $\lambda x. P$ (per variabili e combinatori) induttivamente:

- Se $P = x \rightarrow S K K$
- Se $x \notin \text{var}(P) \rightarrow K P$
- Se $P = P_1 P_2 \rightarrow S (\lambda x. P_1) (\lambda x. P_2)$

Teorema: per ogni P, Q , si ha $(\lambda x. P) Q \rightarrow^* P[Q/x]$.

Quindi la logica combinatoria è Turing-equivalente al λ -calcolo.

6. Ricorsione e combinatori di punto fisso

Le funzioni nel λ -calcolo sono anonime, quindi la ricorsione diretta non è possibile.

Si usano **combinatori di punto fisso**:

- **Combinatore di Turing:**
 $\Theta = (\lambda x. \lambda y. y (x x y)) (\lambda x. \lambda y. y (x x y))$
 Proprietà: $\Theta E \rightarrow_{\beta} E (\Theta E)$
- **Combinatore di Curry:**
 $Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$
 Proprietà: $Y E \rightarrow_{\beta} E (Y E)$

Questi combinatori permettono di definire funzioni ricorsive nel modo seguente:

data una funzione ricorsiva $f = \dots f \dots$, la si esprime come $F = \lambda f. \dots f \dots$, e si pone $f = \Theta F$.

7. Rappresentazione dei numeri (Church numerals)

I numeri naturali sono rappresentati come iteratori:

- $0 = \lambda s. \lambda z. z$

- $1 = \lambda s. \lambda z. s \ z$
- $2 = \lambda s. \lambda z. s \ (s \ z)$
- $n = \lambda s. \lambda z. s^n \ z$

Operazioni aritmetiche definibili:

- $\text{succ} = \lambda n. \lambda s. \lambda z. s \ (n \ s \ z)$
- $\text{add} = \lambda m. \lambda n. \lambda s. \lambda z. m \ s \ (n \ s \ z)$
- $\text{mult} = \lambda m. \lambda n. \lambda s. m \ (n \ s)$
- $\text{exp} = \lambda m. \lambda n. n \ m$

Predicati e controllo:

- $\text{test0} = \lambda n. \lambda p. \lambda q. n \ (K \ q) \ p$
Restituisce p se $n = 0$, altrimenti q .

8. Calcolabilità e ricorsione generale

- **Tesi di Church-Turing:** le funzioni calcolabili con un metodo effettivo sono esattamente quelle λ -definibili, Turing-calcolabili o ricorsive generali.
- **Funzioni ricorsive primitive:**
 - Base: zero, successore, proiezioni.
 - Chiuse per composizione e ricorsione primitiva.
 - Non includono tutte le funzioni calcolabili (es. Ackermann).
- **Ricorsione generale:** aggiunge l'operatore di minimizzazione (μ), che cerca il primo k tale che $h(k, x_1, \dots, x_m) = 0$.

Codifica nel λ -calcolo:

- Ogni funzione ricorsiva primitiva è definibile con Church numerals.
- La ricorsione generale si implementa con un combinatore di punto fisso che genera la sequenza dei valori di h fino a trovare zero.

9. Sistemi di tipi

I tipi servono a prevenire paradossi e garantire terminazione.

Tipi semplici

- Tipi base: $\tau, \sigma, \rho, \dots$
- Tipo funzione: $\tau \rightarrow \sigma$ (associativo a destra).

Due approcci

- **Church-style:** i tipi sono esplicitati nei termini.
Esempio: $\lambda x:\tau. x:\tau \rightarrow \tau$
- **Curry-style:** i tipi sono inferiti tramite regole.
Data una base Γ di assunzioni di tipo per variabili, si derivano giudizi $\Gamma \vdash E : \tau$.

Regole di inferenza (Curry):

- Se $x:\tau \in \Gamma \rightarrow \Gamma \vdash x : \tau$
- Se $\Gamma \cup \{x:\tau\} \vdash E : \sigma \rightarrow \Gamma \vdash \lambda x.E : \tau \rightarrow \sigma$
- Se $\Gamma \vdash E_1 : \tau \rightarrow \sigma$ e $\Gamma \vdash E_2 : \tau \rightarrow \Gamma \vdash E_1 E_2 : \sigma$

Proprietà

- **Strong normalizzazione:** ogni termine tipato (senza punti fissi) termina.
- **Isomorfismo di Curry-Howard:**
 - Tipi $\leftrightarrow$ formule logiche (implicazione).
 - Termini $\leftrightarrow$ dimostrazioni.
 - β -riduzione $\leftrightarrow$ eliminazione del taglio (modus ponens).

Punti fissi e ricorsione

I combinatori di punto fisso **non sono tipabili**. Per introdurre ricorsione si aggiungono costanti F_τ di tipo $(\tau \rightarrow \tau) \rightarrow \tau$ con regola δ : $F_\tau E \rightarrow_\delta E (F_\tau E)$

10. Conclusioni e proprietà generali

- Il λ -calcolo è un modello di computazione universale.
- La sua sintassi semplice e la potenza espressiva lo rendono ideale per studiare calcolabilità, logica e linguaggi di programmazione.
- I sistemi di tipi garantiscono proprietà di terminazione e correttezza, ma limitano l'espressività (es. nessuna ricorsione non tipata).
- La confluenza assicura che il risultato di una computazione, se esiste, è unico indipendentemente dalla strategia.